



Name: Cohort/Batch: Date: Score:

CQRS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Architecture

TOTAL: 10 QUESTIONS

Q1 What is CQRS and what problem in Architecture does it solve? **itsolve2**

Q6 How does CQRS handle reliability and high availability? **servability**

Q2 What are the core components or concepts in CQRS? **Architecture**

Q7 What observability does CQRS provide out of the box? **Lifecycle**

Q3 How does CQRS compare to its main alternatives? **Configuration**

Q8 What are common performance bottlenecks in CQRS? **Compliance**

Q4 How do you get started with CQRS for a new project? **Hardening**

Q9 What security best practices apply when running CQRS? **Testing**

Q5 What configuration options most affect CQRS's behavior in production? **ssin / IdP**

Q10 What mistakes do teams commonly make when adopting CQRS? **Tuning**



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 CQRS is a tool or platform that

Mitigation

CQRS is a tool or platform that automates or simplifies a specific concern in the Architecture domain, reducing manual effort and operational complexity for engineering teams.

A6 CQRS provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

Observability

A2 CQRS has key building blocks that work

Primitives

CQRS has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 CQRS exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

Information

A3 CQRS trades off simplicity against feature richness

Hardening

CQRS trades off simplicity against feature richness differently than its alternatives. Choose CQRS when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

Performance

A4 Install CQRS via its package manager or

Gotchas

Install CQRS via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run CQRS with the principle of least

Assessment

Run CQRS with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

Concurrency

A10 Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.

Configuration